

Tutorial Backend con Django, PostgreSQL y Docker

Este tutorial explica paso a paso cómo construir una aplicación backend sencilla utilizando Django, PostgreSQL y Docker. Además, uso de ORM de Django y pruebas con Postman.

Requisitos previos:

- Python instalado
- Docker instalado
- Visual Studio Code o editor
- Postman instalado
- DBeaver (opcional para probar base de datos)
- Django lo instalaremos después e incluye ORM

1) Levantamiento de base de datos

Dentro de la carpeta raíz de tu proyecto en este caso llamada **tutorial** crea el archivo `docker-compose.yml` que permite automatizar la creación y ejecución de contenedores Docker, lo utilizaremos para levantar PostgreSQL. Escribe el siguiente código (incluye comentarios explicativos):

```
services:
  db:
    image: postgres:17                # usar versión 17 de postgres
    container_name: db_postgres      # nombre del contenedor

    environment:
      POSTGRES_DB: tutorial_db        # define usuario, contraseña y base de datos
      POSTGRES_USER: usuario
      POSTGRES_PASSWORD: 123456

    ports:
      - "5434:5432"                  # usar puerto 5434 de PC y 5432 del contenedor

    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:                    #preserva los datos después de apagar contenedor
```

Abre Docker Desktop y en una terminal ejecuta el comando `docker compose up -d` el cual inicia todos los servicios definidos en tu archivo, si no hay mensajes de error puedes verificar que la base de datos funcione correctamente utilizando herramientas como DBeaver.

```
PS C:\Users\JUAN\Desktop\tutorial> docker compose up -d
[+] up 2/2
✓ Network tutorial_default Created
✓ Container db_postgres Started
```

2) Crear proyecto Django y conectar con Postgres

Primero crea una carpeta dentro de la carpeta raíz que almacenará todo el backend en nuestro caso simplemente la llamaremos backend por lo que tendremos la estructura `tutorial/backend/` seguido abre una terminal y dentro de la carpeta backend ejecuta el comando `python -m venv venv` y después el comando `venv\Scripts\activate` esto para crear un entorno virtual y evitar conflictos de dependencias.

```
PS C:\Users\JUAN\Desktop\tutorial> cd backend
PS C:\Users\JUAN\Desktop\tutorial\backend> python -m venv venv
PS C:\Users\JUAN\Desktop\tutorial\backend> venv\Scripts\activate
(venv) PS C:\Users\JUAN\Desktop\tutorial\backend>
```

Ahora ejecuta los comandos `pip install django` y `pip install psycopg[binary]` (driver de postgres), seguido del comando `django-admin startproject config .` para empezar el proyecto (el `.` indica instalar todo en la carpeta actual)

```
(venv) PS C:\Users\JUAN\Desktop\tutorial\backend> django-admin startproject config .
```

Después de ejecutar el comando se crearán automáticamente varios archivos busca el archivo `config/settings.py` dentro de él hay una sección llamada **DATABASES** reemplazala por:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'tutorial_db',
        'USER': 'usuario',
        'PASSWORD': '123456',
        'HOST': 'localhost',
        'PORT': '5434',
    }
}
```

Lo anterior se hace porque Django usa por defecto SQLite y así lo cambiamos a Postgres y ponemos los datos necesarios para conectar la DB. Ejecuta el comando `python manage.py migrate` que aplica los cambios estructurales a la base de datos. Ahora Postgres y Django están conectados.

```
(venv) PS_C:\Users\JUAN\Desktop\tutorial\backend> python manage.py migrate
Applying auth.0001_initial... OK
Applying admin.0001_initial... OK
Applying admin.0002_logentry_remove_auto_add... OK
Applying admin.0003_logentry_add_action_flag_choices... OK
```

3) Desarrollo de la aplicación

Un proyecto Django trabaja con múltiples módulos independientes llamados apps. Vamos a crear una app llamada **products** con el comando `python manage.py startapp products`, Ve al archivo `config/settings.py` y busca la sección **INSTALLED_APPS** y agrega **products** así:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'products',
]
```

Si no registras la app Django no la reconoce.

En Django un modelo representa una tabla en PostgreSQL como una clase Python ve al archivo `products/models.py` y crea una tabla así:

```
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=100) # texto máximo 100 caracteres
    price = models.DecimalField(max_digits=10, decimal_places=2) #2 decimales
    stock = models.IntegerField() # Número entero
    created_at = models.DateTimeField(auto_now_add=True) #Fecha automática

    def __str__(self):
        return self.name # Texto que representa el objeto
```

Ejecuta el comando `python manage.py makemigrations` (detecta cambios DB) y despues `python manage.py migrate` (ejecuta la migracion, crea la tabla)

```
(venv) PS C:\Users\JUAN\Desktop\tutorial\backend> python manage.py makemigrations
No changes detected
(venv) PS C:\Users\JUAN\Desktop\tutorial\backend> python manage.py makemigrations
Migrations for 'products':
  products\migrations\0001_initial.py
    + Create model Product
(venv) PS C:\Users\JUAN\Desktop\tutorial\backend> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, products, sessions
Running migrations:
  Applying products.0001_initial... OK
```

4) Endpoints

Vamos a crear un primer endpoint hola mundo, ve a `productos/view.py` y escribe:

```
from django.http import JsonResponse # Importa respuestas en formato JSON

def hello(request): #recibe request
    return JsonResponse({ # Retorna una respuesta JSON
        "message": "Hola mundo Django"
    })
```

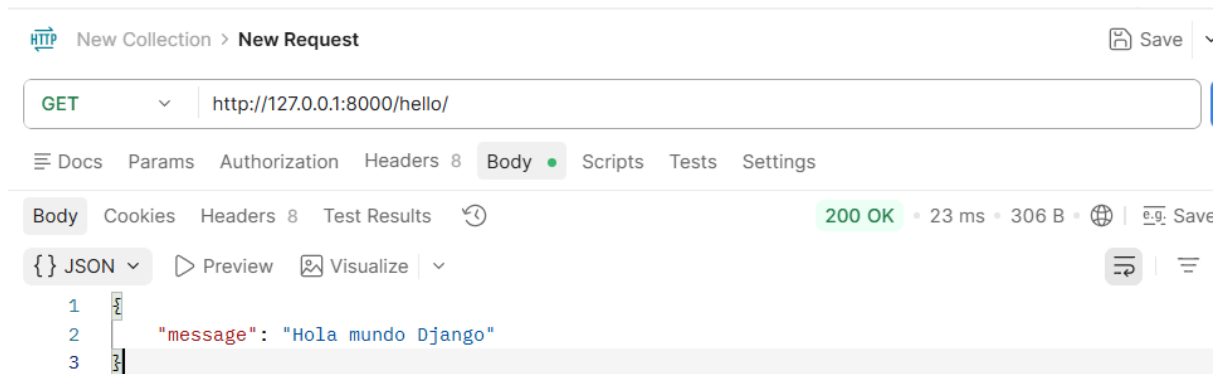
Para conectar la url con su función ve a `config/urls.py` y escribe

```
from django.contrib import admin
from django.urls import path, include
from products.views import hello

urlpatterns = [
    path('admin/', admin.site.urls),

    path('hello/',hello),
]
```

Con el servidor corriendo con el comando `python manage.py runserver` pruébalo en Postman



Ahora vamos a hacer un endpoint para obtener todos los productos en la tabla pero primero añade unos registros antes de empezar, puede ser con DBeaver. Después ve a `products/views.py` y escribe:

```
from django.views.decorators.csrf import csrf_exempt
from django.http import JsonResponse
from .models import Product
import json

@csrf_exempt # Desactiva la protección CSRF (falla sin esto)
def product_list(request):

    products = Product.objects.all() # Obtiene todos los productos de DB
    data = [] # Lista que guarda los productos

    for product in products:
        data.append({
            "id": product.id,
            "name": product.name,
            "price": float(product.price),
            "stock": product.stock,
        })

    return JsonResponse(data, safe=False) # Retorna lista en formato JSON
```

Conecta la url con la función, ve a `config/urls.py` y escribe

```
from products.views import hello, product_list
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),

    path('hello/',hello),

    path('products/',product_list)
]
```

Pruébalo en postman, recuerda arrancar el servidor.

The screenshot shows the Postman interface. At the top, it says 'New Collection > New Request'. The request method is 'GET' and the URL is 'http://127.0.0.1:8000/products/'. The 'Send' button is visible. Below the request bar, there are tabs for 'Docs', 'Params', 'Authorization', 'Headers', 'Body', 'Scripts', 'Tests', and 'Settings'. The 'Body' tab is selected. On the right, there are buttons for 'Save', 'Share', and 'Send'. Below the tabs, the response is displayed. It shows a status of '200 OK', a time of '62 ms', and a size of '332 B'. The response body is in JSON format, showing a list of products. The first product is a monitor with an id of 4, name of 'Monitor', price of 950.0, and stock of 7.

```
{
  "id": 4,
  "name": "Monitor",
  "price": 950.0,
  "stock": 7
}
```

Vamos a crear un endpoint para añadir productos, ve a [products/views.py](#) y añade:

```
@csrf_exempt
def create_product(request):

    if request.method == 'POST': # Verifica que la petición sea POST

        body = json.loads(request.body) #Convierte el body JSON a diccionario

        product = Product.objects.create(
            name=body['name'], # Guarda datos
            price=body['price'],
            stock=body['stock']
        )

        return JsonResponse({
            "message": "Producto creado",
            "id": product.id
        })

    return JsonResponse({
        "error": "Método no permitido"
    }, status=405)
```

Ahora ve a [config/urls.py](#) y añade:

```
from products.views import hello, product_list, create_product
from django.contrib import admin
from django.urls import path, include

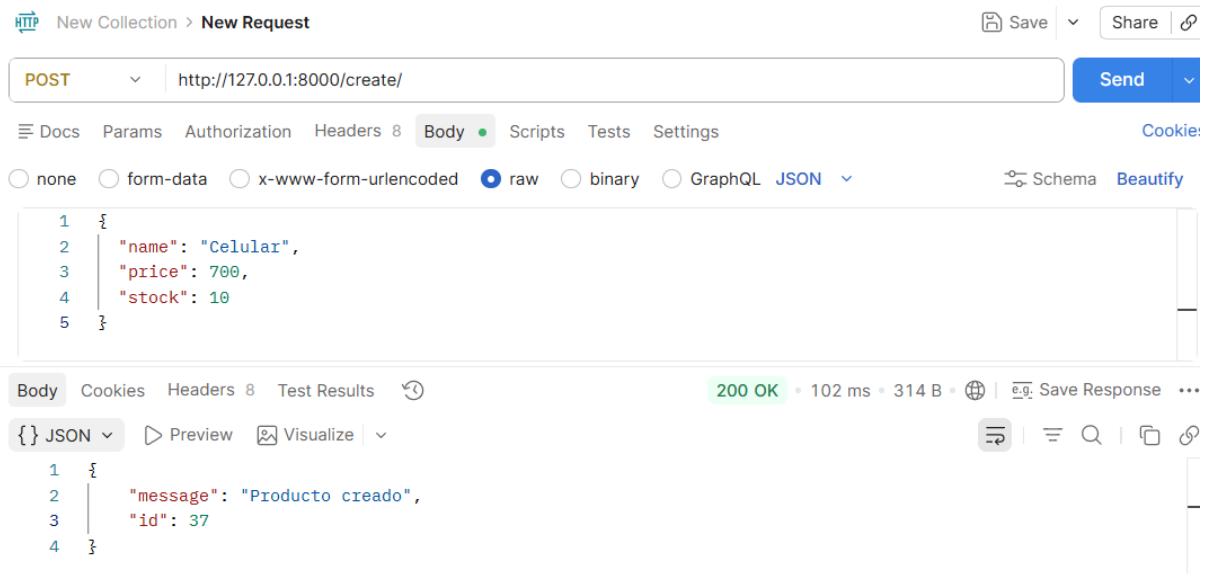
urlpatterns = [
    path('admin/', admin.site.urls),

    path('hello/', hello),

    path('products/', product_list),

    path('create/', create_product)
]
```

Prueba con Postman (pon los datos a insertar en el body y usa método POST):



Ahora puedes adaptar esto a tu aplicación y seguir desarrollando 👍.

